

CSA0613 DESIGN ANALYSIS OF ALGORITHMS

CO3_ASSESSMENT TOOL 1_

Evaluate Binary Search efficiency in large-scale database indexing scenarios by simulating indexed lookups. Record execution time for varying dataset sizes. Analyze how indexing improves search performance. Interpret findings and justify its use in database systems.

Aim

To evaluate the efficiency of Binary Search in large-scale database indexing by simulating indexed lookups, recording execution time for different dataset sizes, analyzing the impact of indexing on search performance, and justifying its use in database systems.

Objective

- To understand Binary Search in indexed databases.
- To compare execution time for different dataset sizes.
- To analyze how indexing improves search speed.
- To interpret the performance results.
- To justify the importance of Binary Search in database indexing.

Problem Statement

Large databases contain millions of records. Searching each record sequentially requires significant time. Database indexing stores data in sorted order, allowing Binary Search to locate records quickly. This assessment evaluates how Binary Search performs as the dataset size increases.

Algorithm

1. Create sorted datasets of different sizes.
2. Select a target value.
3. Apply Binary Search on the indexed dataset.
4. Record the execution time.
5. Repeat for larger datasets.
6. Compare the execution times.
7. Analyze the efficiency.

Python Program

```
1  import time
2  def binary_search(arr, key):
3      low = 0
4      high = len(arr) - 1
5
6      while low <= high:
7          mid = (low + high) // 2
8
9          if arr[mid] == key:
10             return mid
11         elif arr[mid] < key:
12             low = mid + 1
13         else:
14             high = mid - 1
15     return -1
16 sizes = [1000, 10000, 100000, 1000000]
17 print("Dataset Size\tExecution Time (seconds)")
18 for size in sizes:
19     data = list(range(size))
20     key = size - 1
21     start = time.perf_counter()
22     binary_search(data, key)
23     end = time.perf_counter()
24     print(size, "\t\t", end - start)
```

OUTPUT

```
Dataset Size      Execution Time (seconds)
1000              4.62999742012471e-06
10000             4.339999577496201e-06
100000            1.2420001439750195e-05
1000000           1.6639998648315668e-05
```

```
=== Code Execution Successful ===|
```

Time Complexity

Case	Complexity
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$
Space Complexity	$O(1)$

Space Complexity

$O(1)$

Results

- Execution time increases only slightly as dataset size grows.
- Binary Search requires very few comparisons.
- Indexed searching is much faster than linear searching.
- Large databases benefit greatly from indexing.
- Search performance remains efficient even with millions of records.